Chomsky Normal Form Converter

Scooter Computer

The following is a conversion of a context-free grammar (CFG) into Chomsky Normal Form (CNF).

This conversion has five steps:

1. Ensure the start variable is not on the RHS
2. Eliminate ϵ-rules by first finding nullable variables
3. Eliminate unit rules
4. RHS is either one terminal or just variables
5. Ensure RHS has at most two variables

Notation: Right-hand side = RHS, Left-hand side = LHS

Visual Guide: Rules in yellow are being modified in this step. Rules in dark yellow/orange were just added. Rules shown in gray have just been removed.

Original Grammar

Let's start with this context-free grammar (CFG):

$$S \to AaB \mid b \mid S$$
$$A \to S \mid \varepsilon \mid AB$$
$$B \to bbb \mid ASA$$

Our goal is to convert this grammar into **Chomsky Normal Form (CNF)**, which means every production must look like:

- $A \to a$ (one variable produces one terminal), or
- $A \to BC$ (one variable produces exactly two variables)

Why CNF? CNF is useful because it gives us a standardized form that makes parsing algorithms (like the CYK algorithm) much more efficient. Think of it like converting fractions to a common denominator—it makes everything easier to work with!

1 Step 1: Create a New Start Variable

The Problem: In the original grammar, S appears on the right-hand side of some productions. In CNF, the start variable should never appear on the RHS—it should only be used to "kick off" derivations.
The Solution: Create a brand new start variable S_0 (read as "S sub-zero" or "S naught") that only points to the old start variable:

$$\begin{aligned}
\mathbf{S_0} &\to \mathbf{S} \\
S &\to AaB \mid b \mid S \\
A &\to S \mid \varepsilon \mid AB \\
B &\to bbb \mid ASA
\end{aligned}$$

Think of it like this: S_0 is a "wrapper" around our original grammar. It keeps S protected and ensures we can safely modify the rest of the grammar.

2 Step 2: Eliminate ε-Rules (Epsilon Rules)

2.1 Step 2a: Find Nullable Variables

What's a nullable variable? A variable is *nullable* if it can eventually produce the empty string ε (epsilon). This can happen directly (like $A \to \varepsilon$) or indirectly (like $A \to B$ and $B \to \varepsilon$).
In our grammar, these variables are nullable:

$$V_\varepsilon = \{A\}$$

Why does this matter? Anywhere these variables appear, they might "disappear" (become ε). We need to account for all possibilities.

2.2 Step 2b: Analyze Rules Containing Nullable Variables

Now we examine *every* rule in the grammar to see which ones contain nullable variables. For each rule that contains nullable variables, we need to generate new rules covering all possible cases where those variables "disappear."

Analysis of $S \to AaB$

Original Rule:
$S \to A\,aB$

Nullable Variables in this rule: $\{A\}$

New Rules Generated:

- Remove $\{A\}$: $S \to aB$

Analysis of $A \to AB$

Original Rule:
$A \to A\,B$

Nullable Variables in this rule: $\{A\}$

New Rules Generated:

- Remove $\{A\}$: $A \to B$

Analysis of $B \to ASA$

Original Rule:
$B \to A\,S\,A$

Nullable Variables in this rule: $\{A, A\}$

New Rules Generated:

- Remove $\{A\}$: $B \to SA$
- Remove $\{A\}$: $B \to AS$
- Remove $\{A, A\}$: $B \to S$

Understanding the analysis:

- Variables highlighted in yellow are nullable
- Each box shows an original rule and all the new rules we must add
- "Remove $\{A\}$" means create a version where A disappears (becomes ε)
- If multiple nullable variables exist, we try all combinations

2.3 Step 2c: Add New Rules for Each Possibility

For every production that contains a nullable variable, we create additional versions where that variable is "removed" (replaced with nothing). If multiple nullable variables appear, we consider all combinations.
Example: If A is nullable and we have $A \to ABC$, we add the production $A \to BC$ (where A vanished).
After adding all these new possibilities:

$$
\begin{aligned}
S &\to S \mid aB \mid AaB \mid b \\
A &\to S \mid AB \mid B \mid \varepsilon \\
B &\to ASA \mid SA \mid S \mid AS \mid bbb \\
S_0 &\to S
\end{aligned}
$$

Notice the rules in yellow contain nullable variables, and the rules in dark yellow/orange are the newly added possibilities!

2.4 Step 2d: Delete All ε-Rules

Now that we've covered all the cases, we can safely remove every production that directly produces ε (shown in gray above):

$$
\begin{aligned}
S &\to S \mid aB \mid AaB \mid b \\
A &\to S \mid AB \mid B \\
B &\to ASA \mid SA \mid S \mid AS \mid bbb \\
S_0 &\to S
\end{aligned}
$$

✓ **Eliminate ε-rules**

3 Step 3: Eliminate Unit Rules

3.1 Step 3a: What's a Unit Rule?

What's a unit rule? A **unit rule** looks like $A \to B$, where both A and B are single variables (not terminals like a or b).
Why are they problematic? They create unnecessary "middlemen" in derivations. For example:

$$A \to B$$
$$B \to abc \mid xyz$$

We can skip the middleman: just let $A \to abc \mid xyz$ directly!

3.2 Step 3b: Visualize the Dependencies

To systematically find all the "middlemen," we draw a graph where each arrow represents a unit rule:

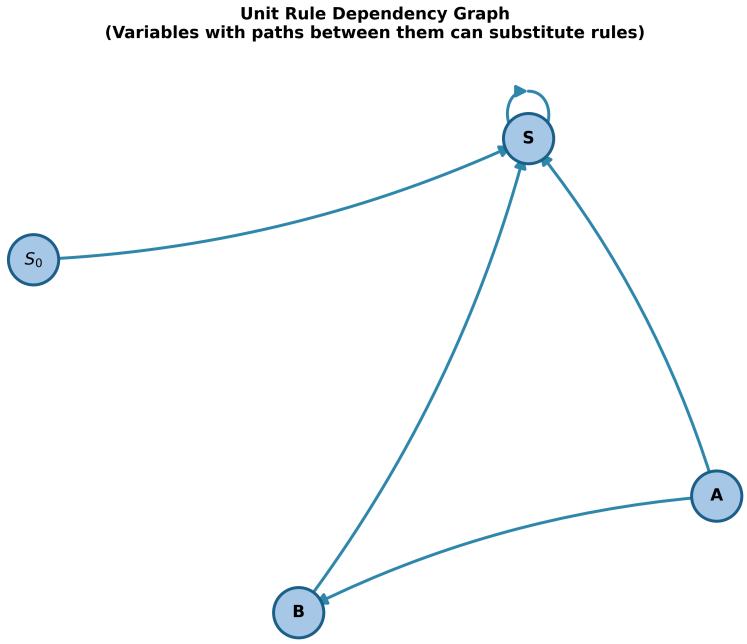

Figure 1: **Unit Rule Dependency Graph.** Each arrow $A \to B$ means there's a unit rule from A to B. If you can follow arrows from X to Y, then X can "borrow" all of Y's non-unit productions.

3.3 Step 3c: Follow All Paths (Using Floyd-Warshall)

We use the **All-Pairs Shortest Path (APSP)** algorithm (specifically Floyd-Warshall) to find every pair of variables with a path between them.

How it works: For each pair (X, Y):

- If distance $= \infty \to$ No path exists $\to$ Don't add anything
- If distance is finite $\to$ Path exists $\to$ Add all non-unit rules from Y to X

Here's what we find by following the paths:

Path Discovery #1

From A to S:

- **Path:** $A \to S$
- **Distance:** 1 step
- **Action:** Add these productions to A: $aB \mid AaB \mid b$

Path Discovery #2

From A to B:

- **Path:** $A \to B$
- **Distance:** 1 step
- **Action:** Add these productions to A: $ASA \mid SA \mid AS \mid bbb$

Path Discovery #3

From B to S:

- **Path:** $B \to S$
- **Distance:** 1 step
- **Action:** Add these productions to B: $aB \mid AaB \mid b$

Path Discovery #4

From S_0 to S:

- **Path:** $S_0 \to S$
- **Distance:** 1 step
- **Action:** Add these productions to S_0: $aB \mid AaB \mid b$

3.4 Step 3d: Remove the Unit Rules Themselves

After "copying over" all the necessary productions, we delete the unit rules themselves (shown in gray below). The dark yellow/orange rules were added from following unit rule paths:

$$
\begin{aligned}
S &\to aB \mid AaB \mid b \\
A &\to AB \mid \text{ASA}\mid \text{AS}\mid \text{SA}\mid \text{aB}\mid \text{AaB}\mid \text{bbb}\mid \text{b} \\
B &\to \text{AaB}\mid ASA \mid SA \mid \text{aB}\mid AS \mid bbb \mid \text{b} \\
S_0 &\to \text{aB}\mid \text{AaB}\mid \text{b}
\end{aligned}
$$

✓ **Eliminate unit rules**

4 Step 4: Separate Terminals from Variables

4.1 Step 4a: The CNF Requirement

In CNF, if a production has length greater than 1, it must contain *only* variables—no terminals allowed. Productions like these violate this rule:

$$A \to aBC \quad \textbf{X} \text{ Mixes terminal } a \text{ with variables } B \text{ and } C$$

4.2 Step 4b: The Fix — Create Helper Variables

The Fix: For each terminal that appears alongside variables, we create a new variable that "stands in" for that terminal:

$$\begin{gathered} U_a \to a \\ U_b \to b \\ \text{etc.} \end{gathered}$$

Then we can rewrite $A \to aBC$ as $A \to U_aBC$.

4.3 Step 4c: After Applying This Transformation

Rules with mixed terminals and variables are replaced with rules containing only variables. New helper variables U_x are introduced for each terminal:

$$\begin{aligned} S &\to b \mid U_aB \mid AU_aB \\ A &\to AB \mid ASA \mid AS \mid SA \mid b \mid U_aB \mid AU_aB \mid U_bU_bU_b \\ B &\to ASA \mid SA \mid AS \mid b \mid U_aB \mid AU_aB \mid U_bU_bU_b \\ S_0 &\to b \mid U_aB \mid AU_aB \\ [Ua] &\to a \\ [Ub] &\to b \end{aligned}$$

✓ RHS is either one terminal or all variables

5 Step 5: Break Up Long Productions

5.1 Step 5a: The Final CNF Requirement

Productions with variables must have *exactly two* variables on the RHS. No more, no less. Productions like these violate this rule:

$$A \to BCD \quad \textbf{X} \text{ Has three variables}$$

5.2 Step 5b: The Fix — Introduce Intermediate Variables

The Fix: We break long productions into chains of two-variable productions:

$$A \to BCD \quad \text{becomes} \quad A \to BE \text{ and } E \to CD$$

We process from left to right, pairing variables and creating intermediates as needed. If a pairing already exists in the grammar, we reuse it!

5.3 Step 5c: Final Grammar in Chomsky Normal Form

$$
\begin{aligned}
S &\to b \mid U_aB \mid CB \\
A &\to AB \mid AS \mid SA \mid b \mid U_aB \mid DA \mid CB \mid EU_b \\
B &\to SA \mid AS \mid b \mid U_aB \mid DA \mid CB \mid EU_b \\
S_0 &\to b \mid U_aB \mid CB \\
[Ua] &\to a \\
[Ub] &\to b \\
C &\to AU_a \\
D &\to AS \\
E &\to U_bU_b
\end{aligned}
$$

✓ **RHS has at most two variables**

Verification: Is This Really CNF?

Let's check our work! Every production in the grammar above has one of these forms:

Form	Description
$A \to a$	Single variable produces single terminal
$A \to BC$	Single variable produces exactly two variables

Success! This grammar is in Chomsky Normal Form and is equivalent to the original grammar.

What can we do with CNF? Now that our grammar is in CNF, we can:

- Use the **CYK (Cocke-Younger-Kasami) algorithm** to parse strings efficiently
- Easily determine if a string belongs to the language in $O(n^3)$ time
- Analyze the grammar's structure more systematically